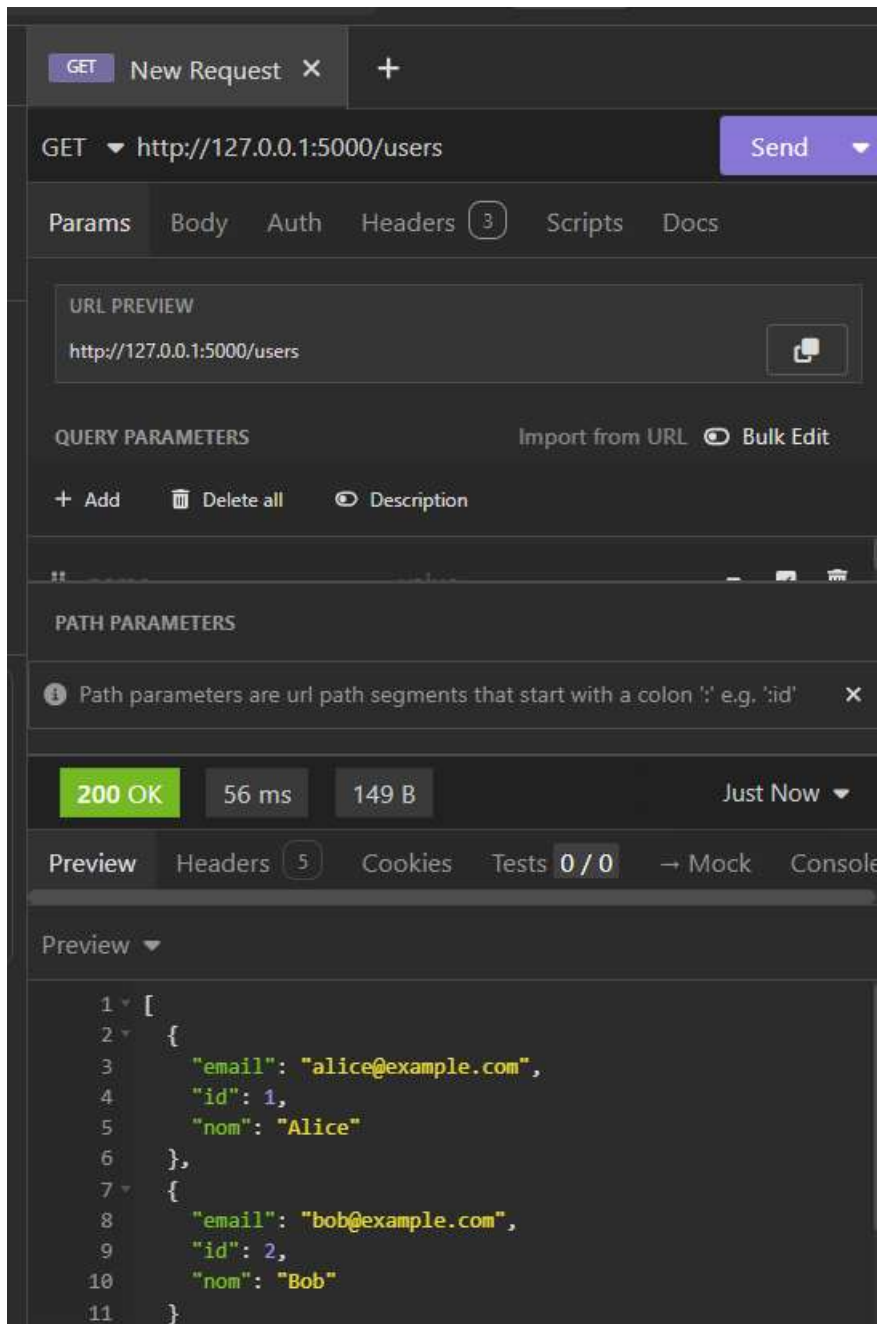
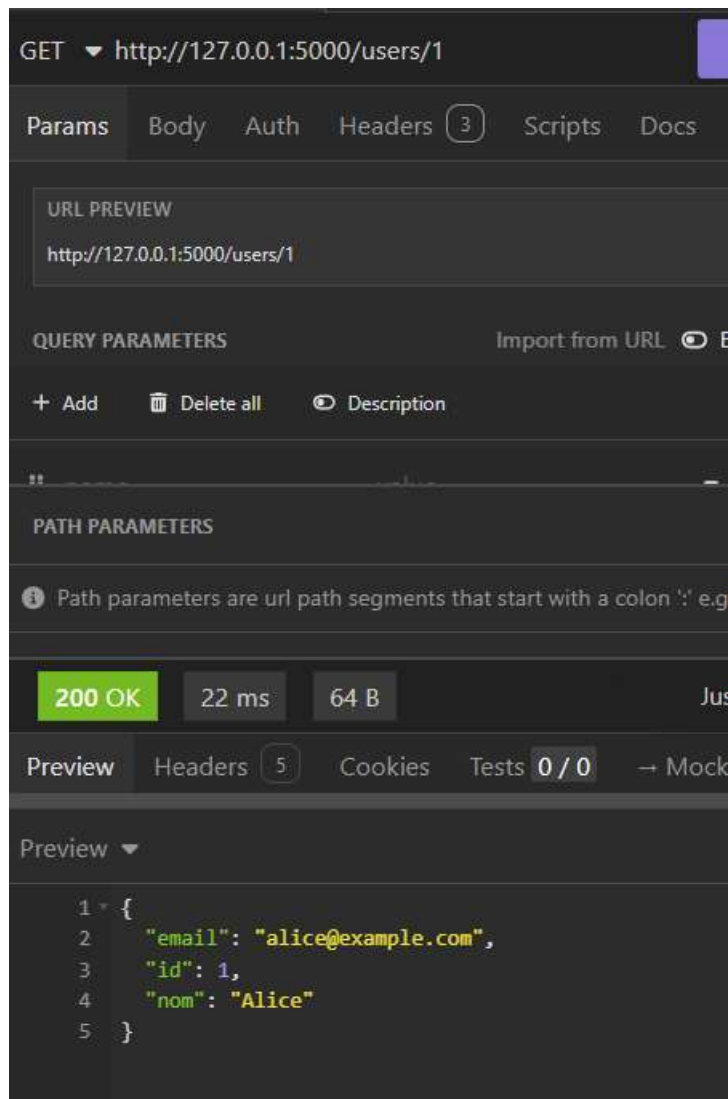


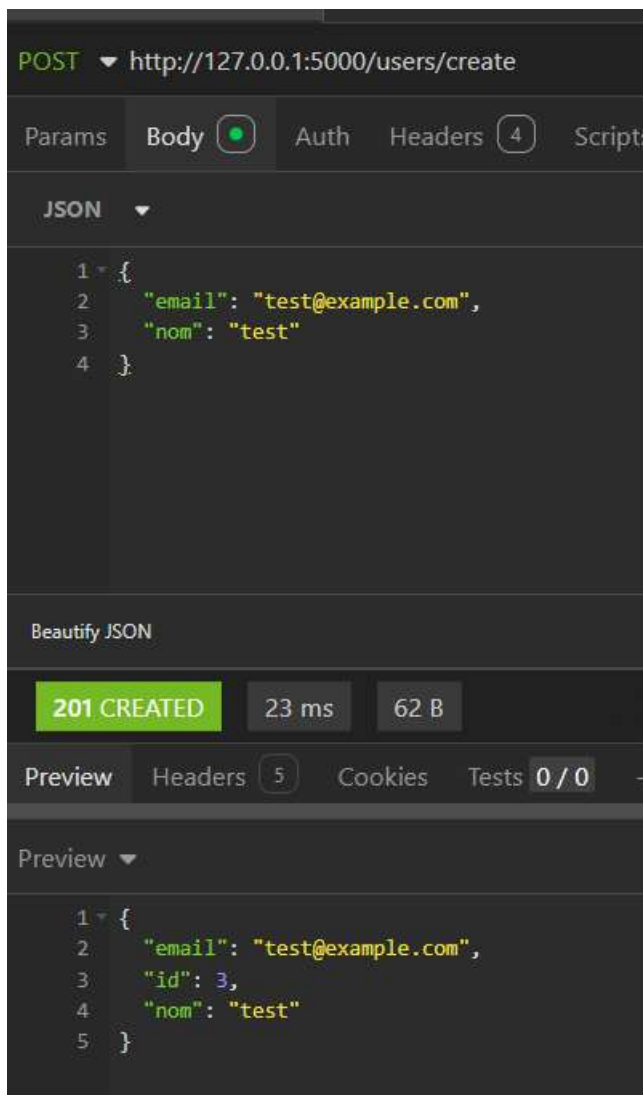
Méthode GET pour récupérer tous les utilisateurs :



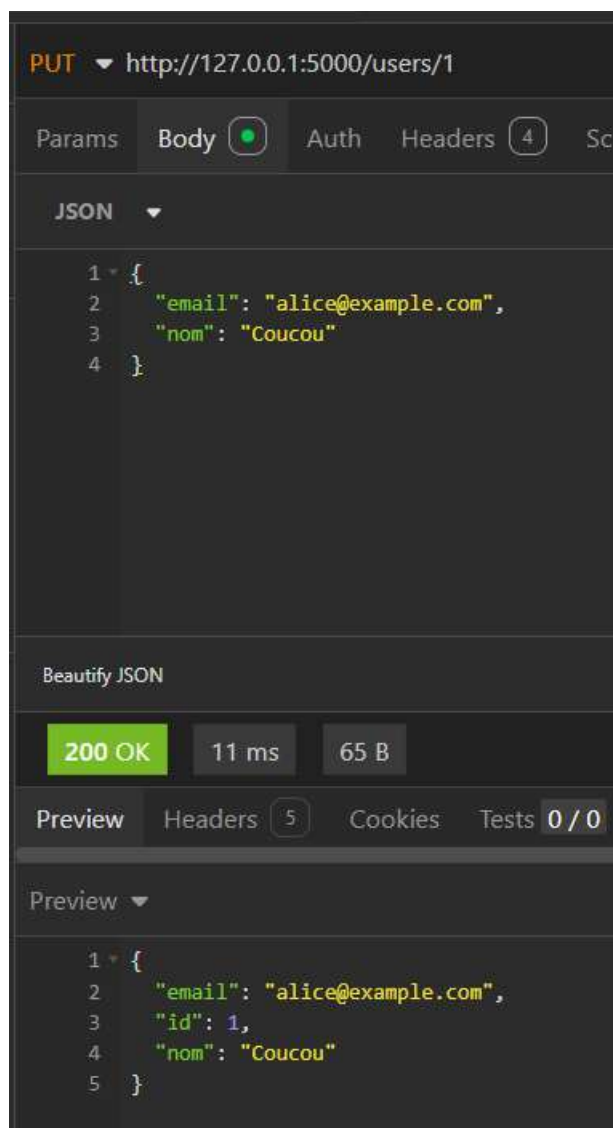
Méthode GET pour récupérer 1 utilisateur spécifique :



Méthode POST pour crée un utilisateur (réservé aux administrateurs) :



Méthode PUT pour mettre à jour les informations d'un utilisateur :



Méthode DELETE pour supprimer un utilisateur :

The screenshot displays a REST client interface with a dark theme. At the top, the method **DELETE** is selected, and the URL is `http://127.0.0.1:5000/users/delete/1`. Below the URL bar, tabs for **Params**, **Body**, **Auth**, **Headers** (with a count of 3), and **Scripts** are visible. The **URL PREVIEW** section shows the same URL. The **QUERY PARAMETERS** section includes an **Import from URL** button and controls for **Add**, **Delete all**, and **Description**. A table with columns **name** and **value** is present but empty. The **PATH PARAMETERS** section has an information icon and a description: "Path parameters are url path segments that start with a colon". The response status is **200 OK** in a green box, with **28 ms** and **40 B** displayed next to it. Below the status bar, tabs for **Preview**, **Headers** (with a count of 5), **Cookies**, and **Tests** (with a count of 0/0) are shown. The **Preview** tab is active, showing a JSON response:

```
1 {
2   "message": "User 1 supprimé"
3 }
```

Méthode POST pour récupérer un JWT :

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:5000/login
- Body:** JSON

```
1 {  
2   "nom": "Alice",  
3   "password": "1234"  
4 }
```
- Status:** 200 OK
- Time:** 32 ms
- Size:** 349 B
- Response Type:** JSON
- Preview:**

```
1 {  
2   "token":  
3     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcmVzaCI6ZmFsc2UsIm1hdCI6MTc4MzQyNDE0OCwianRpIjoiMmQ1MjZhdGAtNmY0NC00MTg0LTkwNjYtZDNiZnQ1ODE3ODUyIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6IksawNlIiwibmJmIjoxNzgzNDI0MTQ4LCJjc3JmIjoiaZTI4ZWlxYjAtMDNjMS00YVJkLTk2YjItYTQ0ZmZjNjQ5MGQzIiwiaXhwIjoxNzgzNDI1MDQ4fQ.BQ4ZxrQ-TfrUwtbM2Rz2gWoof-65tzmfa9LWeg0f6CM"  
3 }
```

Méthode POST pour crée un utilisateur en étant authentifié avec JWT:

The screenshot displays a REST client interface for a POST request to `http://127.0.0.1:5000/users`. The 'Auth' tab is active, showing 'Bearer Token' authentication. The 'ENABLED' checkbox is checked, and the 'TOKEN' field contains a masked JWT token. The 'PREFIX' field is empty. The response status is `200 OK` with a response time of `64 ms` and a body size of `68 B`. The 'Preview' tab shows the JSON response:

```
1 {  
2   "email": "charlie@example.com",  
3   "id": 3,  
4   "nom": "Charlie"  
5 }
```

Méthode GET pour tester la connexion en HTTPS :

The screenshot displays a REST client interface with the following components:

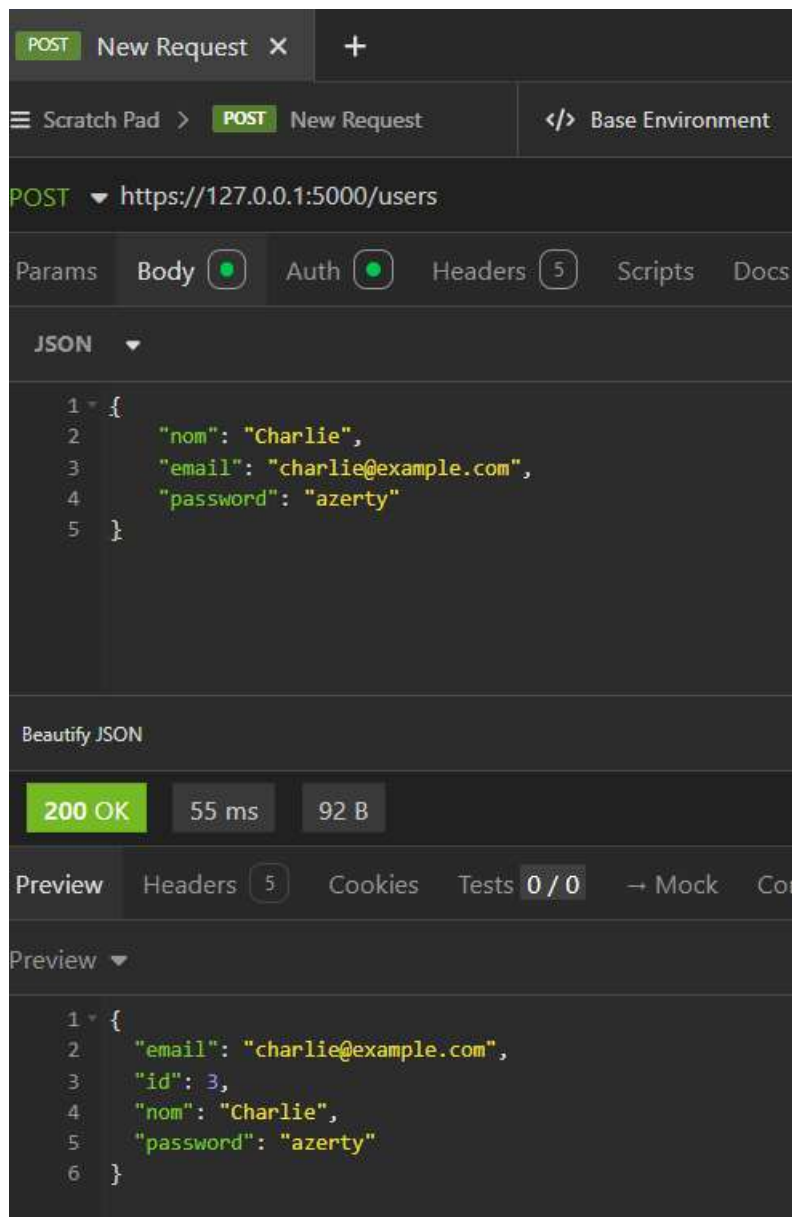
- Request Bar:** Shows the method `GET` and the URL `https://127.0.0.1:5000/users`.
- Auth Tab:** The `Auth` tab is selected, showing a `None` dropdown and a large lock icon with the text "Select an auth type from above".
- Response Status Bar:** Displays `200 OK` in green, `49 ms` in grey, and `197 B` in grey.
- Response Preview:** The `Preview` tab is selected, showing a JSON array of user objects. The first object is expanded, showing the following details:

```
1 [
2   {
3     "email": "alice@example.com",
4     "id": 1,
5     "nom": "Alice",
6     "password": "1234"
7   },
```


Screenshot qui montre que JWT a bien le rôle de la personne :

```
{
  "fresh": false,
  "iat": 1783511297,
  "jti": "cc3c9869-d6b4-49e9-ae81-06a10b108022",
  "type": "access",
  "sub": "Alice",
  "nbf": 1783511297,
  "csrf": "1bc3c8cd-d4f6-4aa0-bc39-6b49d8a26c4b",
  "exp": 1783512197,
  "role": "admin"
}
```

Screenshot qui montre que admin ont le droit sur cette endpoint :



Screenshot qui montre que user n'ont pas le droit sur cette endpoint :

